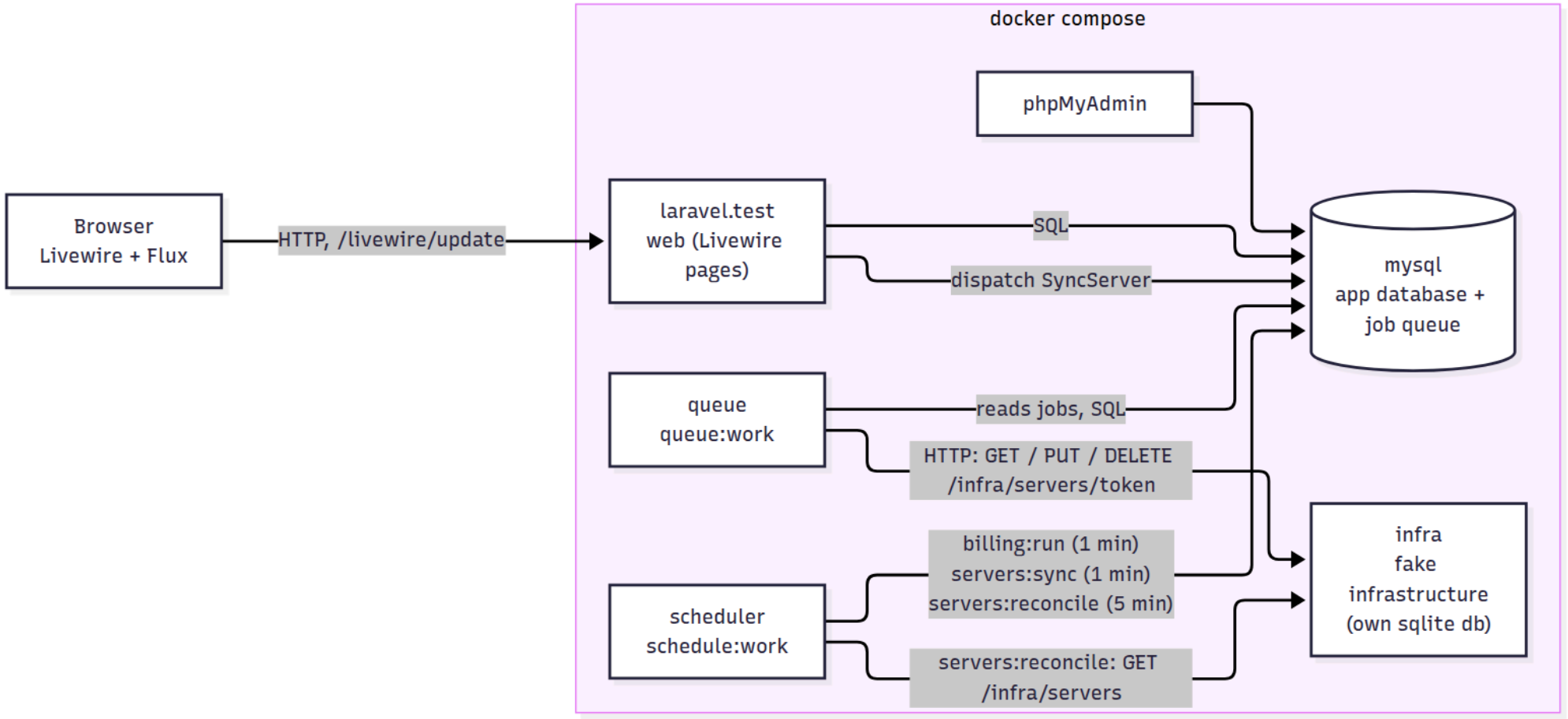
Mini Cloud billing desk

Billing back office of a small cloud provider

- Laravel 12, Livewire 4, Flux UI, MySQL, Sail, one Laravel module per area
- A fake infrastructure that is slow, flaky, and sometimes lies
- What I care about: what happens when things go wrong

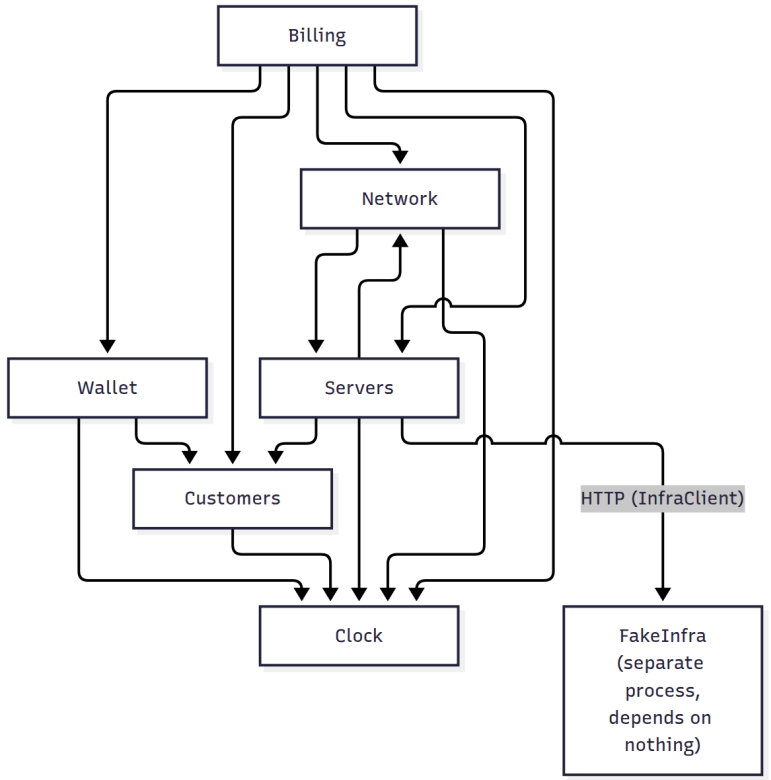
Built: wallets (ledger), servers, IP pool and moves, hourly billing, roles, strict input, admin panel, drift check, price change, catch-up after downtime, one bad record.
Not built: running out of money (notice, suspension, termination) and restore.

The parts and how they talk



Browser and app talk through Livewire. The queue worker is the only one that talks to the infra (plus the drift check in the scheduler). The infra has its own database and knows nothing about the app.

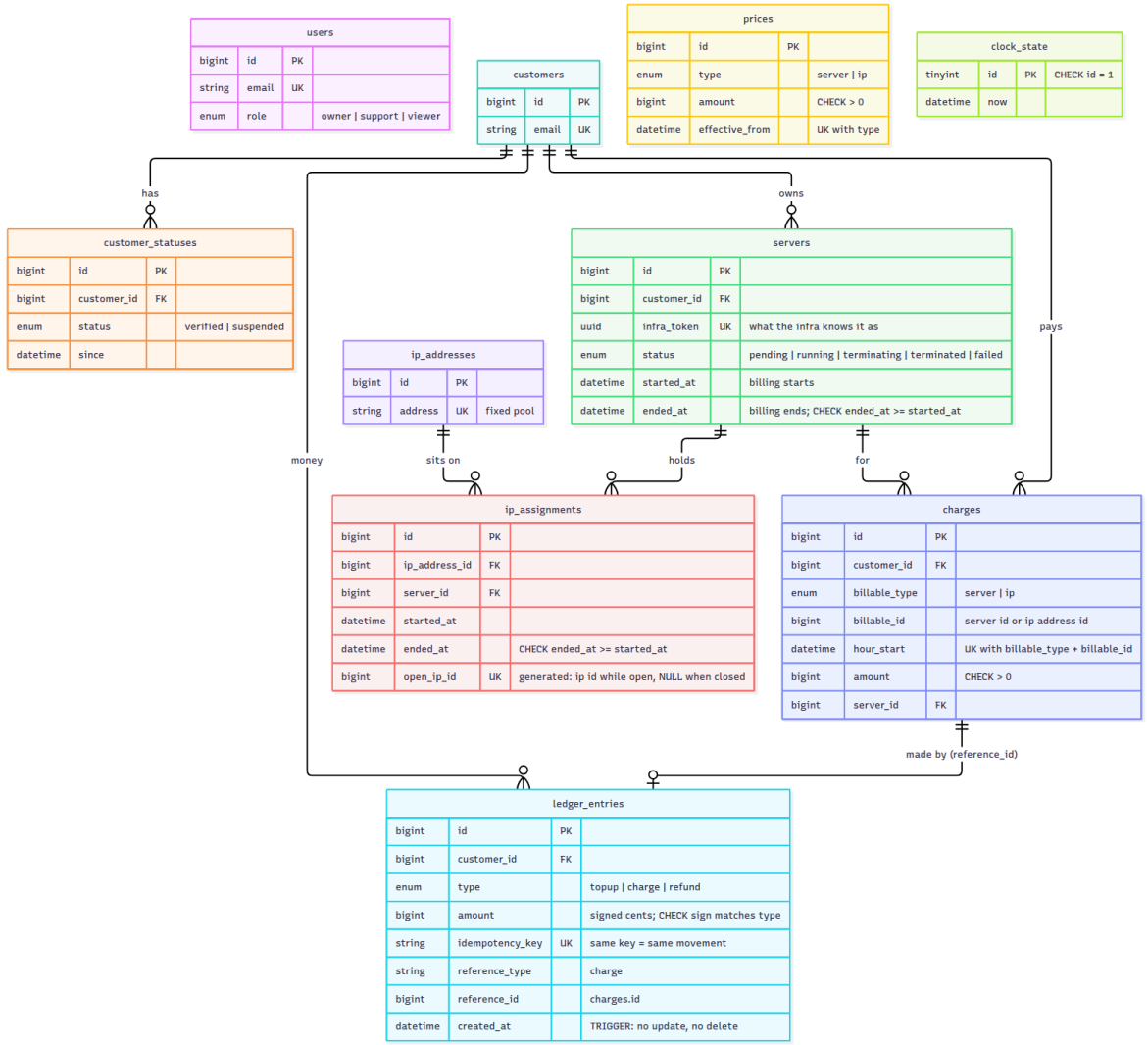
Modules



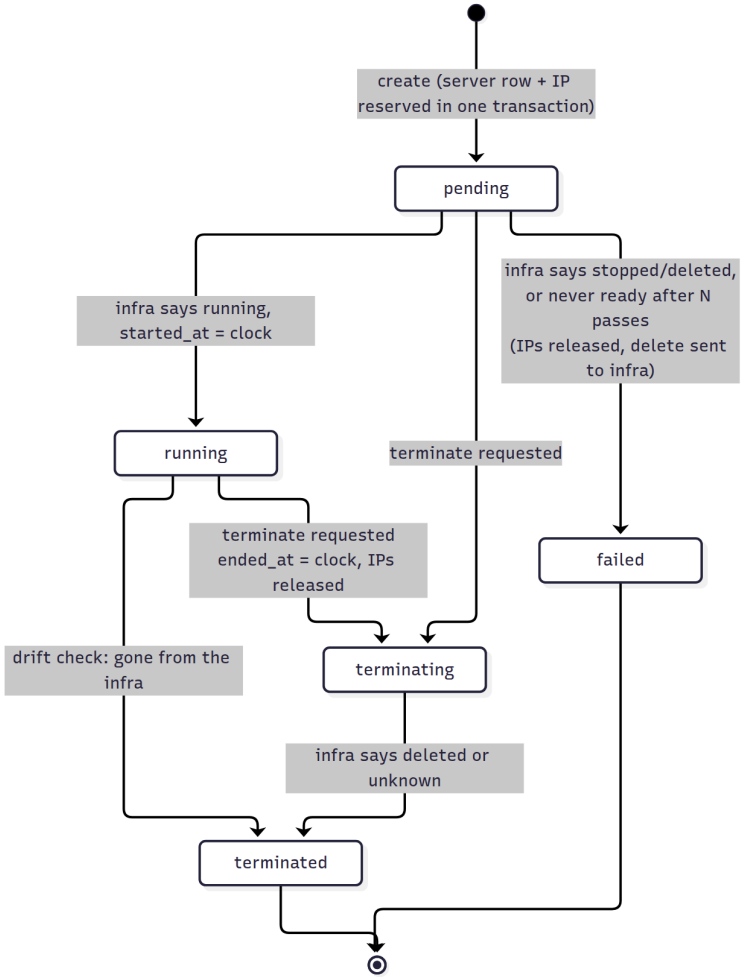
Each module owns its tables, services and Livewire components. The customer page is built from components of other modules (`wallet.customer-ledger`, `servers.customer-servers`).

Honest note: Servers and Network know each other. An interface (`AllocatesIp`) only covers allocation.

Data model (rules the database enforces)

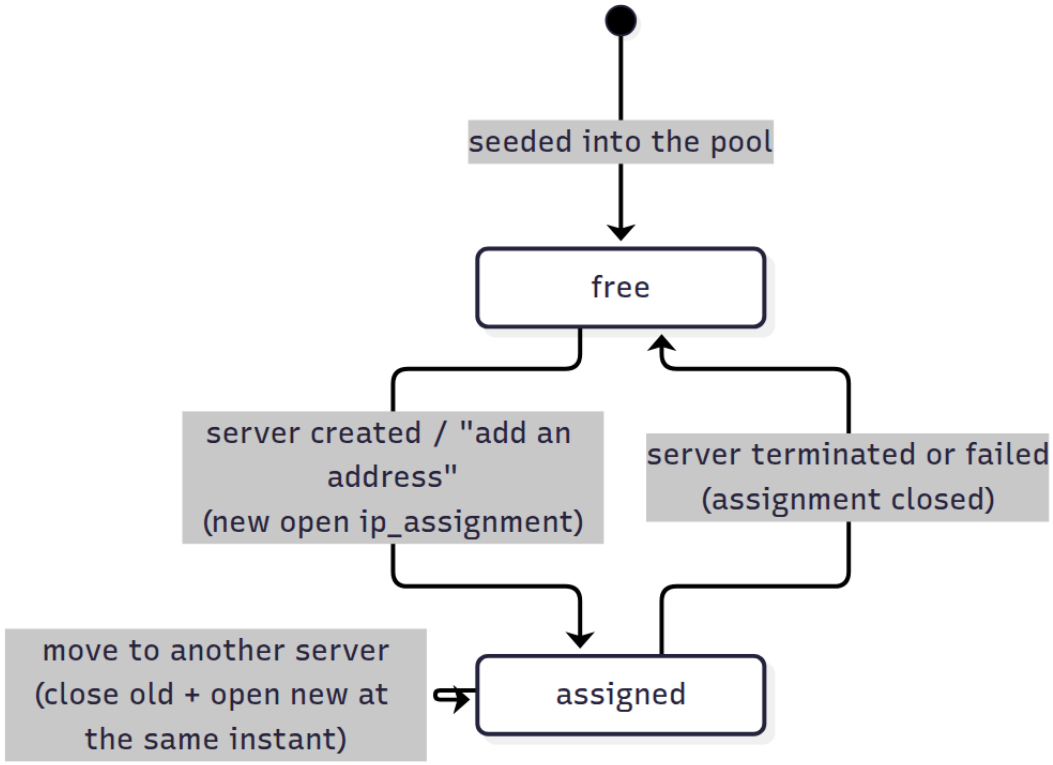


Life cycle of a server



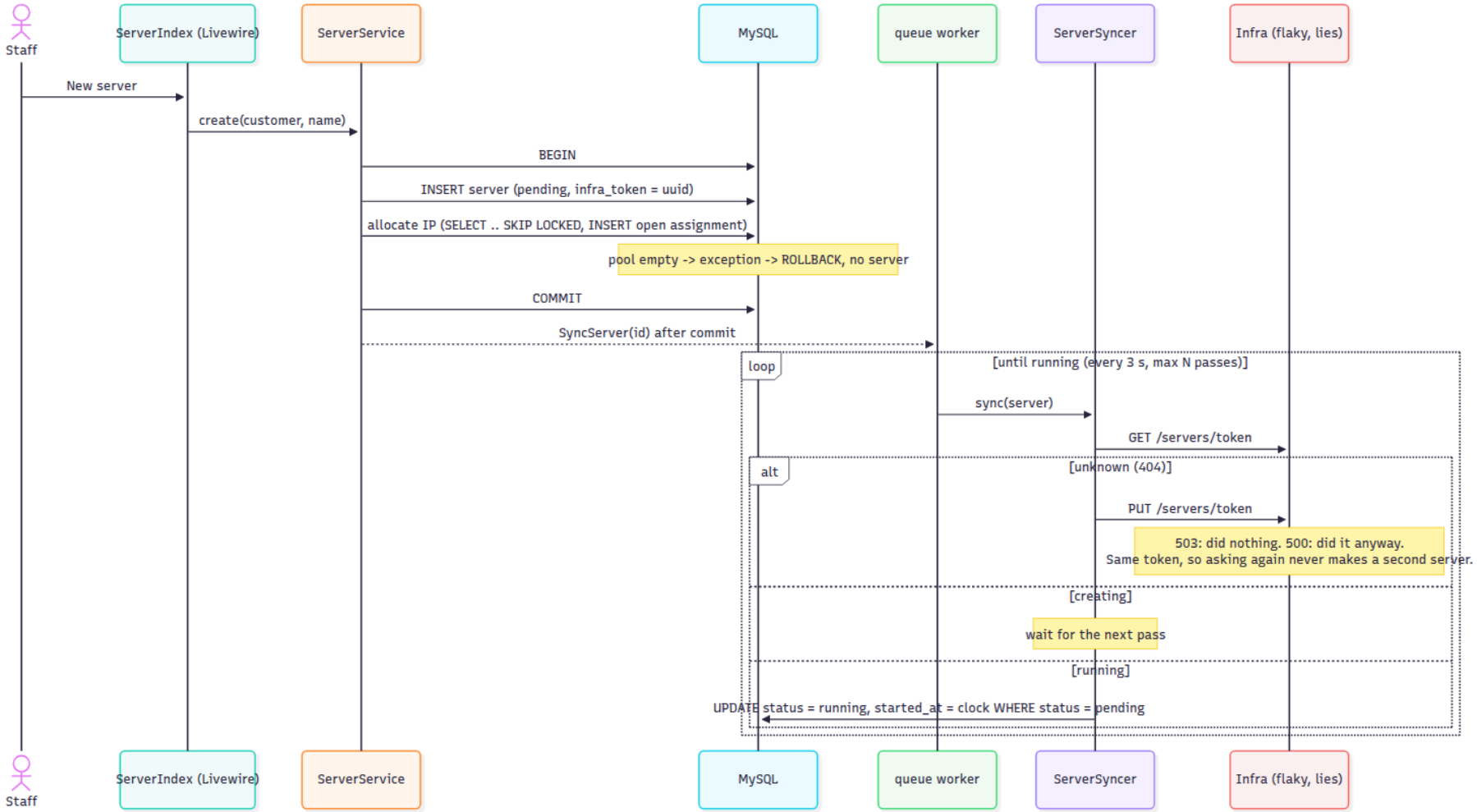
No `started_at`, no billing. Terminating sets `ended_at` and frees the IPs at once, before the infra confirms.

Life cycle of an IP address

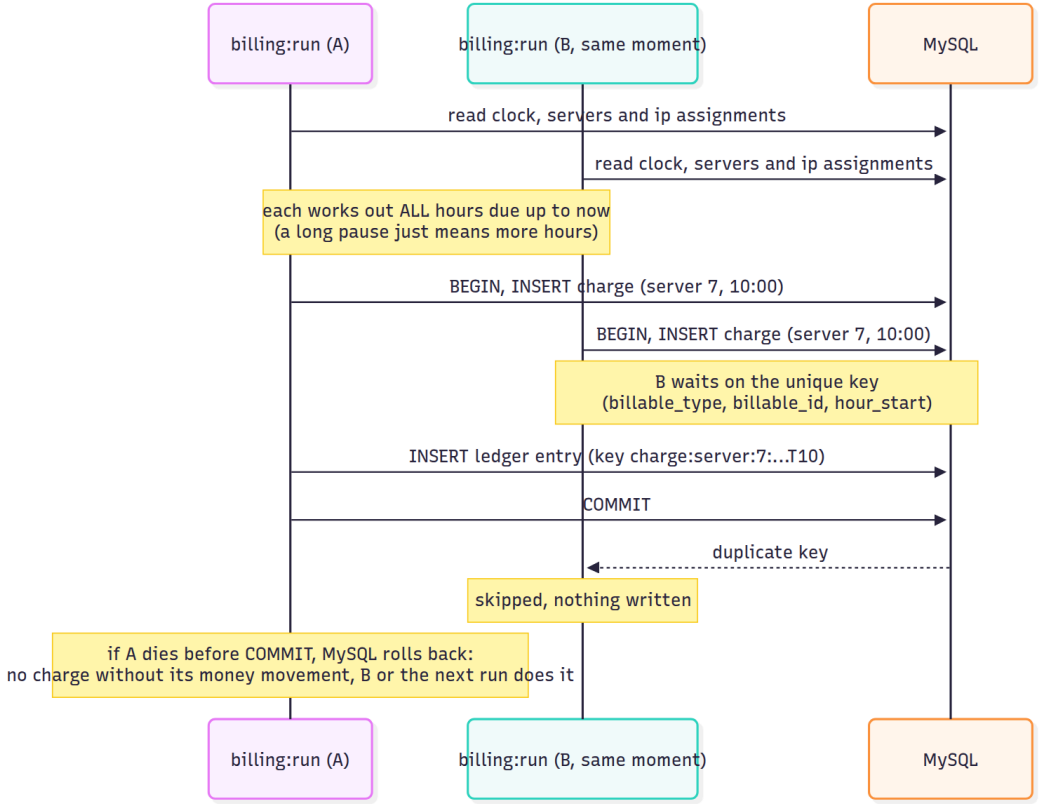


Only one open assignment per address, enforced by a unique index. A move closes the old period and opens the new one at the same instant, in one transaction.

Flow: creating a server

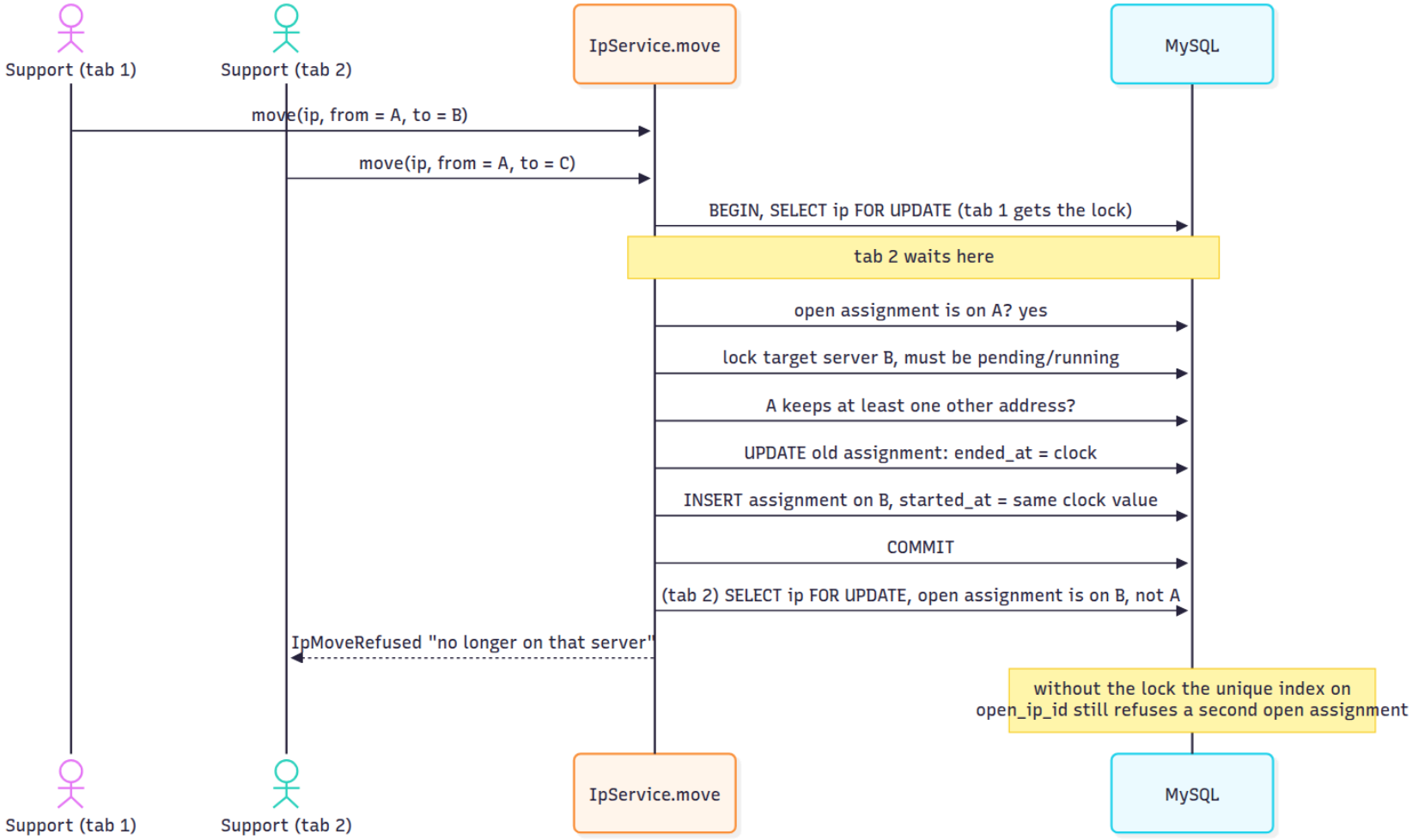


Flow: the billing run

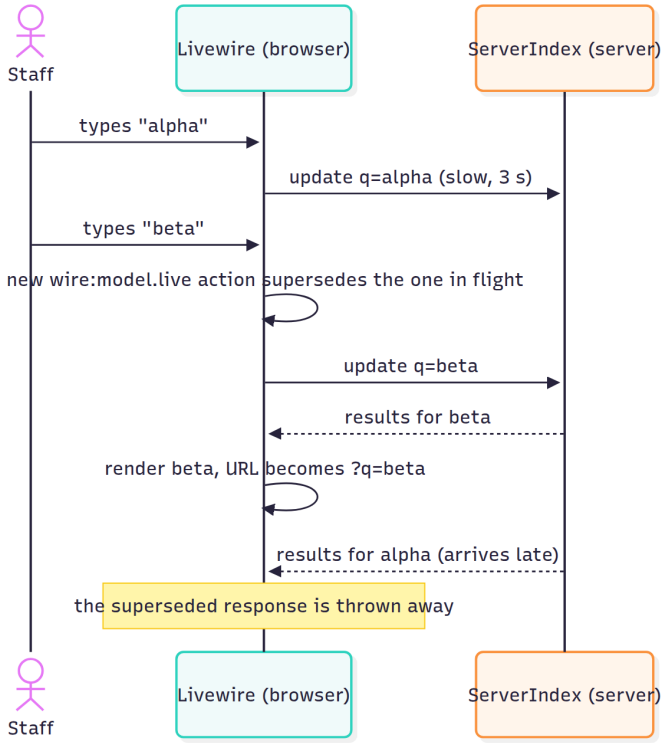


Every run works out all hours due up to the clock. The unique key lets each in once. A pause just means more hours to find.

Flow: moving an IP



Flow: searching in the UI



State is in the URL (`?q=&status=&customer=&sort=&dir=&page=`). Unknown parameters are a 422, bad values too. Loading, empty and failed are three different things on every list.

Decisions, and what I rejected

Decision	Rejected
unique key on `(resource, hour)`, charge + ledger entry in one transaction	a "billed until" cursor
unique index on a generated column (`open_ip_id`) + row lock + expected holder	locking in code only
the infra knows only our token; every call is idempotent	storing the id the infra returns
compare-and-set on status	holding a row lock during an HTTP call
balance = sum of an append-only ledger (triggers)	a balance column
manual clock in the database	real time + offset
Livewire's own request superseding	our own sequence numbers

Challenges and the test that proves each

Challenge	Test
Exactly-once billing	`BillingRaceTest`: 3 runs at the same moment; a run killed with SIGKILL halfway, then another run
One IP, one payer	`IpRaceTest`: two real processes; `IpRulesTest`: database refuses a 2nd open assignment; `BillingRulesTest`: one payer per IP-hour
Stale search results	`tests/Browser/stale-search.mjs`: old answer delayed 3 s, must not replace the new one

Seen failing without the fix: dropping the IP unique index + lock, dropping the charge/ledger unique keys, and `wire:model.live.async` each broke the matching test. Restored afterwards.

Not safe yet, and what next

- The billing run reads every server and address each time: needs a bounded candidate query (the unique key stays the guarantee)
- MySQL can't say "no overlapping closed periods per IP"; only the open one is a constraint
- Drift check ignores `stopped`; giving up counts passes, not time
- Servers and Network know each other: a read model would cut the cycle
- Browser test needs Chrome and a running stack, not part of `php artisan test`
- No audit log, no UI for prices, notice/suspension/restore not built

"We charged them for an IP they removed yesterday"

1. Customer page, money history: the `ip 203.0.113.x <hour>` charges.
2. Server page "Held before", or `ip_assignments` for that address: when did it leave their server?
3. Right if their server held the IP at some moment of that hour and nobody held it earlier in that hour. Removed at 10:30: the 10:00 hour is correct, later hours are not.
4. If they are right: the owner refunds from the customer page. A new ledger entry, the charge stays visible. Nothing is edited.